

Built-in Methods

remove(): is a list method used to delete the **first occurrence of a value** from a list.

Syntax: list.remove(value)

```
x = [10, 20, 30]
x.remove(20)
print(x)
```

[10, 30]

```
x = [10, 20, 30, 20]
x.remove(20)
print(x)
```

[10, 30, 20]

If value not found: ValueError

```
x = [1, 2, 3]
x.remove(5)
```

```
-----
ValueError
Cell In[30], line 2
      1 x = [1, 2, 3]
----> 2 x.remove(5)
```

ValueError: list.remove(x): x not in list

Tuple

```
x = (10, 20, 30)
x.remove(20)
print(x)
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[29], line 2
      1 x = (10, 20, 30)
----> 2 x.remove(20)
      3 print(x)

AttributeError: 'tuple' object has no attribute 'remove'
```

Built-in Methods

pop(): removes an element from a list **using its index** and returns that element.

Syntax: list.pop(index)

```
x = [10, 20, 30, 40]

x.pop(1)
print(x)

[10, 30, 40]
```

Without Index: remove last value

```
x = [10, 20, 30]

x.pop()
print(x)

[10, 20]
```

What about pop(4)

```
: x = [10, 20, 30, 40]

x.pop(4)
```

```
-----
IndexError
Cell In[33], line 3
      1 x = [10, 20, 30, 40]
----> 3 x.pop(4)

IndexError: pop index out of range
```

Negative index

```
x = [10, 20, 30, 40]

x.pop(-1)
print(x)

[10, 20, 30]
```

Tuple

```
t = (1, 2, 3)

t.pop(2)
```

```
-----
AttributeError
Cell In[35], line 2
      1 t = (1, 2, 3)
----> 2 t.pop(2)
```

Built-in Methods

clear(): removes **all elements** from a list.

Syntax: list.clear()

```
x = [1, 2, 3, 4]

x.clear()
print(x)
```

```
[]
```

Tuple

```
t = (1, 2, 3)
t.clear()
```

```
-----
AttributeError
Cell In[37], line 2
      1 t = (1, 2, 3)
----> 2 t.clear()
```

Built-in Methods

index(): is a method used to **find the position (index) of a value**.

Syntax: `list.index(value)`

```
: x = [10, 20, 30, 20]
print(x.index(20))
```

1

Value not present

```
x = [1, 2, 3]
x.index(5)
```

```
-----
ValueError
Cell In[40], line 3
      1 x = [1, 2, 3]
----> 3 x.index(5)
```

Using start and end range

`list.index(value, start, end)`

```
: x = [10, 20, 30, 20, 40]
print(x.index(20, 2, 5))
```

```
x = [10, 20, 30, 20, 40]
print(x.index(20, 2, 5))
```

3

Tuple

```
t = (5, 10, 15)
print(t.index(10))
```

```
t = (5, 10, 15)
print(t.index(10))
```

1

```
x = (10, 20, 30, 20, 40)
print(x.index(20, 2, 5))
```

3

Built-in Methods

count(): is used to find **how many times a value appears** in a sequence.

Syntax: list.count(value)

```
x = [10, 20, 30, 20, 20]
print(x.count(20))
```

Output-3

Value not present

```
x = [1, 2, 3]
print(x.count(5))
```

Output- 0 (No Error)

Tuple

```
t = (1, 2, 2, 3)
print(t.count(2))
```

2

Built-in Methods

sort(): is a list method used to **arrange elements in ascending or descending order**.

Syntax: list.sort()

Ascending Order

```
x = [5, 2, 9, 1]

x.sort()
print(x)

[1, 2, 5, 9]
```

Descending Order

```
x = [5, 2, 9, 1]

x.sort(reverse=True)
print(x)

[9, 5, 2, 1]
```

With String

```
names = ["Riya", "Ankit", "Jay"]

names.sort()
print(names)

['Ankit', 'Jay', 'Riya']
```

Case Sensitive

```
x = ["a", "B", "c"]

x.sort()
print(x)

['B', 'a', 'c']
```

Works only if elements Are comparable

```
: x = [10, "Ankit", 5]
x.sort()

-----
TypeError
Cell In[52], line 2
      1 x = [10, "Ankit", 5]
----> 2 x.sort()
```

Tuple

```
t = (3, 1, 2)
t.sort()

-----
AttributeError
Cell In[53], line 2
      1 t = (3, 1, 2)
----> 2 t.sort()
```

Built-in Methods

reverse(): is a list method used to **reverse the order** of elements in a list.

Syntax: list.reverse()

```
x = [1, 2, 3, 4]

x.reverse()
print(x)

[4, 3, 2, 1]
```

Tuple

```
: t = (1, 2, 3)
  t.reverse()
```

```
-----
AttributeError
Cell In[55], line 2
      1 t = (1, 2, 3)
----> 2 t.reverse()
```

Built-in Functions

sum(): is a built-in function used to add all numeric elements of an iterable (like list, tuple, set).

Syntax: `sum(iterable,start)`

List

```
x = [10, 20, 30]
print(sum(x))
```

60

Tuple

```
x = (10, 20, 30)
print(sum(x))
```

60

With Start value

```
x = [10, 20, 30]
print(sum(x,10))
```

70

Elements must be numeric

```
x = [1, 2, "3"]
sum(x)
```

```
-----
TypeError
Cell In[60], line 3
      1 x = [1, 2, "3"]
----> 3 sum(x)
```

ord(): returns the ASCII value of a character.

Syntax: `ord(character)`

```
: print(ord('A'))
   print(ord('a'))
   print(ord('9'))
```

65

97

57

Built-in Methods

max(): used to find the **largest (maximum)**

value in an iterable or among given values.

Syntax: max(iterable)

```
x = [10, 50, 20, 5]
```

```
print(max(x))
```

```
50
```

min(): used to find the **minimum value** in an iterable or among given values.

Syntax: min(iterable)

```
x = [10, 50, 20, 5]
```

```
print(min(x))
```

```
5
```

Set

Set:

- A set is a collection of unique elements with no fixed order.

Creating a Set:

S={1,2,3,4}

S=set((1,2,3,4))

S=set([1,2,3,4])

S=set("hello")

S=set() – empty set

```
s={5,3,1,2,5,4,3,2,6,7,3,2,5,4,1}
```

```
: s=set(5,3,1,2,5,4,3,2,6,7,3,2,5,4,1)
```

```
-----  
TypeError                                Traceback  
Cell In[37], line 1  
----> 1 s=set(5,3,1,2,5,4,3,2,6,7,3,2,5,4,1)  
  
TypeError: set expected at most 1 argument, got 15
```

- The set() constructor accepts only one argument.
- That argument must be an iterable (list, tuple, string, etc.).

Set

Set:

- A set is a collection of unique elements with no fixed order.

Properties:

- No duplicates

```
s={1,5,3,6,4,3,7,8,3,6,7}
```

```
s
```

```
{1, 3, 4, 5, 6, 7, 8}
```

- Elements must be immutable

```
a={1,2,3,4}
a[0]=10
a
```

```
-----
TypeError
Cell In[20], line 2
```

This will raise an error because sets are unordered and do not support indexing.

- Mutable (can change)

```
a={1,2,3,4}
a.add(100)
a
```

```
{1, 2, 3, 4, 100}
```

```
a.remove(2)
a
```

```
{1, 3, 4, 100}
```

```
: a=[[1,2],3}
a
```

```
-----
TypeError
Cell In[15], line 1
----> 1 a=[[1,2],3}
      2 a
```

```
: a={(1,2),3}
a
```

```
: {(1, 2), 3}
```

Set

Accessing the elements:

Access using for loop

```
s = {10, 20, 30, 40}
```

```
for i in s:  
    print(i)
```

```
40  
10  
20  
30
```

- Access each element one by one
- Order is not guaranteed

Check membership (in operator)

```
s = {10, 20, 30}
```

```
print(20 in s)    # True  
print(50 in s)    # False
```

```
True  
False
```

- Used to test if element exists.

Convert set to list/tuple

```
s = {10, 20, 30}
```

```
l = list(s)  
print(l[0])
```

```
10
```

- Order may change each run.

Set

Delete elements from a set:

Remove()

```
s = {10, 20, 30}

s.remove(20)
print(s)
```

{10, 30}

- Output order may vary

Discard()

```
s = {10, 20, 30}

s.discard(20)
print(s)
```

{10, 30}

- No error if element not present

Pop()

```
s = {10, 20, 30}

x = s.pop()
print(x)    # random value
print(s)
```

10
{20, 30}

- Sets are unordered; we don't know which element is removed.

del

```
: s = {10, 20, 30}
del s
```

- Delete the entire set variable, not just elements

s

```
-----
NameError
Cell In[11], line 1
----> 1 s
```

Delete multiple elements

```
s = {10, 20, 30, 40}

for i in {20, 40}:
    s.discard(i)

print(s)

{10, 30}
```

Set

Frozen set:

- A frozen set is an immutable version of a set.

Creating a Set:

S=frozenset((1,2,3,4))

S=frozenset([1,2,3,4])

S=frozenset("hello")

S=frozenset()

```
S=frozenset((1,2,3,4))
S.add(100)
```

```
-----
AttributeError
Cell In[36], line 2
      1 S=frozenset((1,2,3,4))
----> 2 S.add(100)
```

```
s=frozenset((1,2,3,4))
s.remove(1)
```

```
-----
AttributeError
Cell In[39], line 2
      1 s=frozenset((1,2,3,4))
----> 2 s.remove(1)
```

Set

Mathematical set /frozenset operations:

Union

```
a={1,2,3}  
b={3,4,5}  
a.union(b)
```

{1, 2, 3, 4, 5}

```
a = frozenset([1,2,3])  
b = frozenset([3,4,5])  
a.union(b)  
  
frozenset({1, 2, 3, 4, 5})
```

Difference

```
a={1,2,3}  
b={3,4,5}  
a.difference(b)
```

{1, 2}

```
a = frozenset([1,2,3])  
b = frozenset([3,4,5])  
a.difference(b)
```

frozenset({1, 2})

Intersection

```
a={1,2,3}  
b={3,4,5}  
a.intersection(b)
```

{3}

```
a = frozenset([1,2,3])  
b = frozenset([3,4,5])  
a.intersection(b)
```

frozenset({3})

Symmetric Difference

```
a={1,2,3}  
b={3,4,5}  
a.symmetric_difference(b)
```

{1, 2, 4, 5}

```
a = frozenset([1,2,3])  
b = frozenset([3,4,5])  
a.symmetric_difference(b)
```

frozenset({1, 2, 4, 5})

Exercise

Given 3 sets:

A → Patients with Hypertension

B → Patients with Diabetes

C → Patients with Cardiac Issues

Find:

1. Patients with all three
2. Only cardiac issues
3. Hypertension but not diabetes
4. Exactly two diseases

```
A = {"P1", "P2", "P3", "P4"} # Hypertension
B = {"P3", "P4", "P5", "P6"} # Diabetes
C = {"P2", "P3", "P6", "P7"} # Cardiac
```

1. Patients with all three

```
all_three = A.intersection(B)
all_three_1=all_three.intersection(C)
print("All three diseases:", all_three_1)
```

All three diseases: {'P3'}

```
all_three = A.intersection(B,C)
print("All three diseases:", all_three)
```

All three diseases: {'P3'}

```
all_three = A & B & C
print("All three diseases:", all_three)
```

All three diseases: {'P3'}

2. Only Cardiac issue: Patients in C but not in A or B:

```
: only_cardiac = C.difference(A,B)
print("Only cardiac issues:", only_cardiac)
```

Only cardiac issues: {'P7'}

```
only_cardiac = C - (A | B)
print("Only cardiac issues:", only_cardiac)
```

Only cardiac issues: {'P7'}

Set

```
A = {"P1", "P2", "P3", "P4"} # Hypertension
B = {"P3", "P4", "P5", "P6"} # Diabetes
C = {"P2", "P3", "P6", "P7"} # Cardiac
```

3. Hypertension but not diabetes:

```
htn_not_diabetes = A.difference(B)
print("Hypertension but not Diabetes:", htn_not_diabetes)
```

Hypertension but not Diabetes: {'P1', 'P2'}

```
htn_not_diabetes = A - B
print("Hypertension but not Diabetes:", htn_not_diabetes)
```

Hypertension but not Diabetes: {'P1', 'P2'}

4. Exactly two diseases:

We find patients in:

- A & B but not C
- A & C but not B
- B & C but not A

```
ab_only = (A.intersection(B)).difference(C)
ac_only = (A.intersection(C)).difference(B)
bc_only = (B.intersection(C)).difference(A)
```

```
exactly_two = ab_only.union(ac_only, bc_only)
```

```
print("Exactly two diseases:", exactly_two)
```

Exactly two diseases: {'P6', 'P4', 'P2'}

```
: ab_only = (A & B) - C
  ac_only = (A & C) - B
  bc_only = (B & C) - A
```

```
exactly_two = ab_only | ac_only | bc_only
```

```
print("Exactly two diseases:", exactly_two)
```

Exactly two diseases: {'P6', 'P4', 'P2'}

Data types: Dictionary

Dictionary:

- A Python dictionary is an unordered collection of items.
- While other compound data types have only one value as an element, a dictionary has a **key: value** pair.
- Dictionaries are optimized to retrieve values when the key is known.
- It is generally used when we have a huge amount of data.

```
my_dict={1:"Apple",2:"Banana", 1:"Mango"}  
my_dict
```

```
{1: 'Mango', 2: 'Banana'}
```

Creating a Dictionary:

- Creating a dictionary is as simple as placing items inside curly braces{} separated by a comma.
- While values can be of any data type and can repeat, keys must be of an immutable type and must be unique.
- If duplicate keys exist → the last value overrides the previous one.

```
# Empty Dictionary  
my_dict={}
```

```
# Dictionary with Integer Keys  
my_dict={1:"Apple", 2:"Ball"}  
print(my_dict)
```

```
{1: 'Apple', 2: 'Ball'}
```

```
# Dictionary with Mixed Keys  
my_dict={"Name":"Shyam", "Age":25}  
print(my_dict)
```

```
{'Name': 'Shyam', 'Age': 25}
```

Data types: Dictionary

Accessing Elements from a Dictionary:

- While indexing is used with other container types to access values, a dictionary uses keys.
- The key can be used either inside the square brackets or with the get method.
- The difference while using get() is that it returns None instead of KeyError, if the key is not found.

```
print(my_dict[1])
```

Mango

```
print(my_dict.get(1))
```

Mango

```
print(my_dict[3])
```

```
-----  
KeyError  
Cell In[64], line 1  
----> 1 print(my_dict[3])
```

```
print(my_dict.get(3))
```

None

Data types: Dictionary

Iterating through a Dictionary:

- Using a for loop, we can iterate through each key in a dictionary.

```
student_details={"Roll_No":101, "Name": "Ram", "Course": "Python", "Grade":"A+"}  
print(student_details)
```

```
for key,value in student_details.items():  
    print(key,":",value)
```

```
Name : Ram  
Course : Data Science  
Marks : 90
```

```
for value in student_details.values():  
    print(value)
```

```
Ram  
Data Science  
90
```

```
for key in student_details:  
    print(key)
```

```
Name  
Course  
Marks
```

```
for key in student_details.keys():  
    print(key)
```

```
Name  
Course  
Marks
```

Data types: Dictionary

Add element in a Dictionary:

- Dictionaries are **mutable**. We can add new items or change the value of existing items using the assignment operator.
- If the key is already present, the value gets updated, else a new key:value pair is added to the dictionary.

```
student_details={"Name": "Ram", "Course": "Python"}  
print(student_details)
```

```
{'Name': 'Ram', 'Course': 'Python'}
```

```
student_details["Course"] = "Data Science"  
print(student_details)
```

```
{'Name': 'Ram', 'Course': 'Data Science'}
```

```
student_details["Marks"] = 90  
print(student_details)
```

```
{'Name': 'Ram', 'Course': 'Data Science', 'Marks': 90}
```

Data types: Dictionary

Delete element from a Dictionary:

- We can remove a particular item in a dictionary by using the method pop(). This method removes an item with the provided key and returns the value.
- All the items can be removed at once using the clear() method.
- We can also use the del keyword to remove individual items or the entire dictionary itself.

```
: student_details={"Roll_No":101, "Name": "Ram", "Course": "Python", "Grade":"A+"}  
print(student_details)  
  
{'Roll_No': 101, 'Name': 'Ram', 'Course': 'Python', 'Grade': 'A+'}
```

```
: student_details.pop("Course")  
print(student_details)  
  
{'Roll_No': 101, 'Name': 'Ram', 'Grade': 'A+'}
```

```
student_details.clear()  
print(student_details)
```

```
{}
```

```
: student_details={"Roll_No":101, "Name": "Ram", "Course": "Python", "Grade":"A+"}  
del student_details
```

```
student_details
```

```
-----  
NameError  
Cell In[93], line 1  
----> 1 student_details
```

Data types: Dictionary

Python Nested Dictionary:

```
: students = {  
    101: {"name": "Alice", "course": "Machine Learning", "GPA": 3.8},  
    102: {"name": "Bob", "course": "Data Science", "GPA": 3.5},  
    103: {"name": "Charlie", "course": "AI", "GPA": 3.9},  
    104: {"name": "Diana", "course": "Cybersecurity", "GPA": 3.6},  
    105: {"name": "Ethan", "course": "Cloud Computing", "GPA": 3.7}  
}
```

```
: students[101]
```

```
: {'name': 'Alice', 'course': 'Machine Learning', 'GPA': 3.8}
```

```
: students[101]={"name": "Ram", "course": "Python programming", "GPA": 4.9}
```

```
: students[101]
```

```
: {'name': 'Ram', 'course': 'Python programming', 'GPA': 4.9}
```

Exercise

Employee Database Management:

- **Create a list of dictionaries representing 5 employees in the company. Each employee's dictionary should contain:**
 - ID: Employee ID (integer)
 - name: Name of the employee (string)
 - department: Department the employee belongs to (string)
 - salary: Salary of the employee (float)

- **add a new employee to the list**
- **remove an employee by their ID**
- **update the salary of an employee based on their ID**

Create a function

- to add a new employee to the list, where information is provided by the user input.
- Remove an employee by their ID, provided by the user.
- Update an employee's salary based on their ID, provided by the user.
- Find an employee by their ID and print their detail; the ID should be the user input.

find an employee by their ID and print their details

Exercise

```
|: employees = [
    {"ID": 101, "name": "John", "department": "HR", "salary": 50000},
    {"ID": 102, "name": "Alice", "department": "IT", "salary": 60000},
    {"ID": 103, "name": "Bob", "department": "Finance", "salary": 55000},
    {"ID": 104, "name": "Diana", "department": "Marketing", "salary": 45000},
    {"ID": 105, "name": "Ethan", "department": "IT", "salary": 70000} ]

def add_employee(employees, new_employee):
    employees.append(new_employee)

def remove_employee(employees, ID):
    for emp in employees:
        if emp["ID"] == ID:
            employees.remove(emp)

def update_salary(employees, ID, new_salary):
    for emp in employees:
        if emp["ID"] == ID:
            emp["salary"] = new_salary
            break

def find_employee_by_id(employees, ID):
    for emp in employees:
        if emp["ID"] == ID:
            print(emp)
```

Exercise

```
: user_input_for_add= input("Enter the details of new employee as ID, name, department, salary seperated by space")
new_employee= list(map(str, user_input_for_add.split()))
new_employee={"ID":int(new_employee[0]), "name":new_employee[1], "department":new_employee[2], "salary":int(new_employee[3])}
new_employee

add_employee(employees, new_employee)

: user_input_for_remove= int(input("Enter the ID of employee to remove"))
remove_employee(employees, user_input_for_remove)

: emp_id, New_salary=int(input("Enter the employee ID: ")), int(input("Enter the new salary: "))
update_salary(employees, emp_id, New_salary)

: emp_id=int(input("Enter the employee ID to search"))
find_employee_by_id(employees, emp_id)
```

Exercise

Movie Database

- List of dictionaries with MovieID, Title, Genre, Rating.

Write functions to:

- Add a new movie
- Remove a movie by ID
- Update movie rating
- List movies of a genre sorted by rating

Dhirubhai Ambani University

(Formerly known as DA-IICT)

Topic: Object Oriented Concepts in Python

Dr. Ankit Vijayvargiya

Assistant Professor

Room No. 4205, Faculty Block 4

Email: [ankit_Vijayvargiya\[at\]dau.ac.in](mailto:ankit_Vijayvargiya[at]dau.ac.in)

Phone: 079-68261628(O), 7877709590(M)

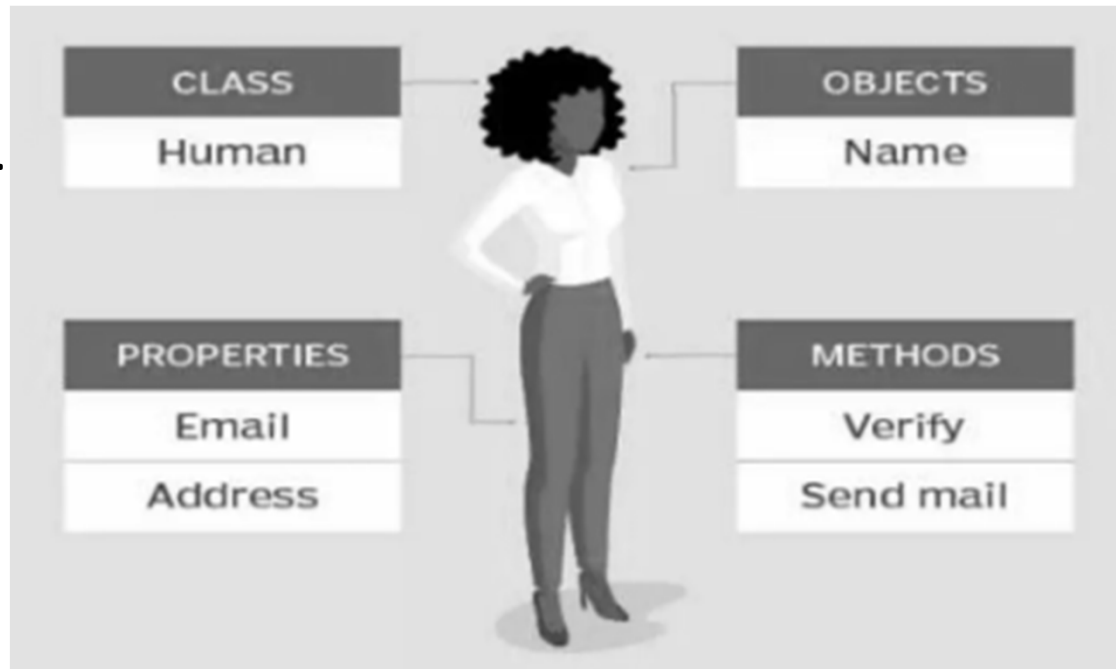
Object Oriented Programming

Paradigm of OOPS



A class is a blueprint for individual objects, attributes, and methods.

Attributes are defined in the class template and represent the state of an object.



Objects are instances of a class created with specifically defined data.

Methods are functions that are defined inside a class that describe the behaviour of the object

Object Oriented Programming

Is Python being OOPS?

- Python is an object-oriented language. This means everything in Python is an object.

```
a=2  
print(type(a))
```

```
a="Python_Programming"  
print(type(a))
```

```
a=[1,2,3,4]  
print(type(a))
```

```
a=2  
print(type(a))
```

<class 'int'>

```
: a="Python_Programming"  
print(type(a))
```

<class 'str'>

```
: a=[1,2,3]  
print(type(a))
```

<class 'list'>

a is the object of the integer class/string class/ list class ➡ Built-in datatypes

Object Oriented Programming

Python Class:

- A class is a **user-defined** data structure that holds its own data members and member functions, which can be accessed and used by creating an instance of that class.
- A class is like a blueprint for an object.

Syntax:

```
class ClassName:  
    # Statements
```

```
obj=ClassName()  
print(obj.attr)
```

- Classes are created by the keyword class.
- Attributes are the variables that belong to a class.
- Attributes are always public and can be assessed using the dot(.) operator.

Example

```
: class vehicle:  
    color= "Black"  
    price=100000  
  
v1=vehicle()  
  
: v1.color  
  
: 'Black'  
  
: v1.price  
  
: 100000
```

Empty Class

```
class vehicle:  
    pass  
v1=vehicle()
```

Memory Management

Memory Management in Python

Memory Management

C-style variable

```
/* C code */  
int a = 5;  
int b = 5;
```

Memory

variable	location	value
a	0x3E8	101
b	0x3E9	101

Memory Management

C-style variable

```
/* C code */  
int a = 5;  
  
// later...  
a = 6
```

Memory

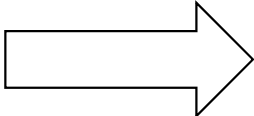
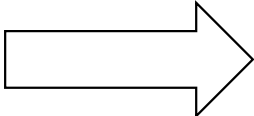
variable	location	value
a	0x3E8	110
b	0x3E9	101

Memory Management

**There are no C-style typed variables in
Python
Or
Python has names, not variables**

Memory Management

How are the Python Objects stored in memory?

Names  **references**  **objects**

A name is just a label for an object.

Memory Management

Different types of Objects:

Simple	Container
Number	dict, set, frozenset
string	List , tuple
	User defined class

- **Simple objects** store their own value
- **Container objects** are capable to store references to simple objects or even other container type objects

Memory Management

What is a Reference?

- A **reference** is basically a way for a name to **point to an object in memory**.

```
sys.getrefcount(a)-1
```

Reference Count

- CPython keeps a reference count for objects.
- New reference → count increases.
- Reference removed → count decreases.
- When the count reaches zero, the object can be deallocated.

Stack/frame → keeps track of execution and references

Heap → contains Python objects

The main benefit of **reference counting** is that Python can know **when an object is no longer being used and can reclaim its memory**.

Object Oriented Programming

Python Class:

- A class is a **user-defined** data structure that holds its own data members and member functions, which can be accessed and used by creating an instance of that class.
- A class is like a blueprint for an object.

Syntax:

```
class ClassName:  
    # Statements
```

```
obj=ClassName()  
print(obj.attr)
```

- Classes are created by the keyword class.
- Attributes are the variables that belong to a class.
- Attributes are always public and can be assessed using the dot(.) operator.

Example

```
: class vehicle:  
    color= "Black"  
    price=100000  
  
v1=vehicle()  
  
: v1.color  
  
: 'Black'  
  
: v1.price  
  
: 100000
```

Empty Class

```
class vehicle:  
    pass  
v1=vehicle()
```

Object Oriented Programming

Python class methods:

Once we have defined the class attributes, we can define functions within the class that can access them and add more functionality to the existing code.

These defined functions inside a class are known as methods.

```
: class vehicle:
:     def vehicle_info(self):
:         self.color="Black"
:         self.price=100000
: v1=vehicle()
:
: v1.vehicle_info()
:
: v1.color
: 'Black'
:
: v1.price
: 100000
```

Self in Python:

- self represents the current object or instance of the class.
- Using the self variable, we can access a class's attributes and methods in Python.

```
: class vehicle:
:     def vehicle_info(self,color,price):
:         self.color=color
:         self.price=price
: v1=vehicle()
:
: v1.vehicle_info("Red", 100000)
:
: v1.color
: 'Red'
:
: v1.price
: 100000
```

Object Oriented Programming

Constructor in Python:

- A constructor in Python is a special method named `__init__` that is automatically called when a new object of a class is created.
- It is used to initialize the object's attribute with specific values at the time of creation.

```
: class vehicle:  
    def __init__(self, color, price):  
        self.color = color  
        self.price = price  
    def show_details(self):  
        print(f"Vehicle Color: {self.color}")  
        print(f"Vehicle Price: {self.price}")  
  
v1=vehicle("Black", 100000)  
v2=vehicle("Red", 50000)
```

```
: v1.show_details()
```

```
Vehicle Color: Black  
Vehicle Price: 100000
```

```
: v2.show_details()
```

```
Vehicle Color: Red  
Vehicle Price: 50000
```

Object Oriented Programming

assert:

- Assert is used to test a condition.
 - If the condition is True : Program continues
 - If the condition is False: Program stops with an error

Syntax: **assert** condition, "Error message"

```
class Vehicle:
    def __init__(self, color, price):
        # Assert validation
        assert price >= 0, f"Price {price} must be >= 0"

        self.color = color
        self.price = price

    def show_details(self):
        print(f"Vehicle Color: {self.color}")
        print(f"Vehicle Price: {self.price}")
```

```
v1 = Vehicle("Black", 100000)
v2 = Vehicle("Red", 50000)

v1.show_details()
v2.show_details()
```

```
Vehicle Color: Black
Vehicle Price: 100000
Vehicle Color: Red
Vehicle Price: 50000
```

```
v1 = Vehicle("Black", -1000)
v2 = Vehicle("Red", 50000)

v1.show_details()
v2.show_details()
```

```
-----
AssertionError
Cell In[17], line 1
----> 1 v1 = Vehicle("Black", -1000)
      2 v2 = Vehicle("Red", 50000)
      4 v1.show_details()
```

The core difference is that if-else controls the normal flow and logic of a program, while assert is used for debugging and validating internal assumptions that should never be false in correct code.

Object Oriented Programming

Class attribute and instance attribute:

Instance Attribute: An attribute that belongs to a specific object (instance) of a class.

- Defined inside methods (usually `__init__`).
- Each object has its own copy.
- Changing it affects only that object.

```
: class Vehicle:
    def __init__(self, color, price):
        self.color = color
        self.price = price
    def show_details(self):
        print(f"Vehicle Color: {self.color}")
        print(f"Vehicle Price: {self.price}")

v1 = Vehicle("Black", 100000)
v2 = Vehicle("Red", 50000)

v1.show_details()
v2.show_details()
```

```
Vehicle Color: Black
Vehicle Price: 100000
Vehicle Color: Red
Vehicle Price: 50000
```

Class Attribute: An attribute that belongs to the class itself, not to individual objects.

- Defined directly inside the class, outside methods.
- Shared by all instances.
- Changing it affects all objects (unless overridden).

```
class Vehicle:
    company_name = "Tata Motors"    # Class attribute
    def __init__(self, color, price):
        self.color = color          # instance attribute
        self.price = price          # instance attribute
    def show_details(self):
        print(f"Company: {self.company_name}")
        print(f"Vehicle Color: {self.color}")
        print(f"Vehicle Price: {self.price}")

v1 = Vehicle("Black", 100000)
v2 = Vehicle("Red", 50000)

v1.show_details()
v2.show_details()
```

```
Company: Tata Motors
Vehicle Color: Black
Vehicle Price: 100000
Company: Tata Motors
Vehicle Color: Red
Vehicle Price: 50000
```

Object Oriented Programming

Class Attribute:

```
class Vehicle:
    company_name = "Tata Motors"    # Class attribute
    def __init__(self, color, price):
        self.color = color          # instance attribute
        self.price = price          # instance attribute
    def show_details(self):
        print(f"Company: {self.company_name}")
        print(f"Vehicle Color: {self.color}")
        print(f"Vehicle Price: {self.price}")
```

```
v1 = Vehicle("Black", 100000)
v2 = Vehicle("Red", 50000)
```

```
v1.show_details()
v2.show_details()
```

```
Company: Tata Motors
Vehicle Color: Black
Vehicle Price: 100000
Company: Tata Motors
Vehicle Color: Red
Vehicle Price: 50000
```

```
: Vehicle.company_name
```

```
: 'Tata Motors'
```

```
: v1 = Vehicle("Black", 100000)
: v1.company_name
```

```
: 'Tata Motors'
```

```
Vehicle.company_name="BMW"
```

```
v1=Vehicle("Black",1000)
v2=Vehicle("Blue",2000)
```

```
v1.show_details()
v2.show_details()
```

```
Company: BMW
Vehicle Color: Black
Vehicle Price: 1000
Company: BMW
Vehicle Color: Blue
Vehicle Price: 2000
```

```
: v1=Vehicle("Black",1000)
: v2=Vehicle("Blue",2000)
```

```
v1.company_name="BMW"
```

```
v1.show_details()
v2.show_details()
```

```
Company: BMW
Vehicle Color: Black
Vehicle Price: 1000
Company: Tata Motors
Vehicle Color: Blue
Vehicle Price: 2000
```

Object Oriented Programming

Object list storage and its representation:

We store objects in a list when we want to:

- Manage multiple objects together
- Perform operations like looping, searching, and sorting
- Build real-world systems (students, vehicles, products, etc.)

```
: v1 = Vehicle("Black", 100000)
v2 = Vehicle("Red", 50000)
v3 = Vehicle("Black", 100000)
v4 = Vehicle("Red", 50000)
v5 = Vehicle("Black", 100000)
v6 = Vehicle("Red", 50000)
```

```
class Vehicle:
    company_name = "Tata Motors"    # Class attribute
    all=[]

    def __init__(self, color, price):
        self.color = color          # instance attribute
        self.price = price          # instance attribute

        Vehicle.all.append(self)

    def show_details(self):
        print(f"Company: {self.company_name}")
        print(f"Vehicle Color: {self.color}")
        print(f"Vehicle Price: {self.price}")
```

```
print(Vehicle.all)
```

```
[<__main__.Vehicle object at 0x00000228F76C2E40>, <__main__.Vehicle object at 0x00000228F5B91F90>, <__main__.Vehicle object at 0x00000228F81C39D0>, <__main__.Vehicle object at 0x00000228F73ED6E0>, <__main__.Vehicle object at 0x00000228F73EF5C0>, <__main__.Vehicle object at 0x00000228F73E3890>]
```

```
for instance in Vehicle.all:
    print(instance.color)
```

Black
Red
Blue
Orange
Green
Red

```
: print(v1)
```

```
<__main__.Vehicle object at 0x000001839B26EBA0>
```



**Not Useful
information**

```
: for instance in Vehicle.all:
    print(f"Color: {instance.color} Price: {instance.price}")
```

Color: Black Price: 100000
Color: Red Price: 50000
Color: Blue Price: 200000
Color: Orange Price: 500000
Color: Green Price: 300000
Color: Red Price: 50000

`__str__` and `__repr__` are both **special methods** in Python that define how an object is represented as a string.

Object Oriented Programming

Object list storage and its representation:

`__str__` : User-Friendly representation

```
class Vehicle:
    company_name = "Tata Motors" # Class attribute
    all = []

    def __init__(self, color, price):
        self.color = color # Instance attribute
        self.price = price # Instance attribute

        Vehicle.all.append(self)

    def show_details(self):
        print(f"Company: {self.company_name}")
        print(f"Vehicle Color: {self.color}")
        print(f"Vehicle Price: {self.price}")

    def __str__(self):
        return f"(Vehicle Color: {self.color}, vehicle cost: {self.price})"

v1=Vehicle("Black",100000)
v2=Vehicle("Red",200000)
v3=Vehicle("Brown",50000)
```

```
print(v1)
```

```
(Vehicle Color: Black, vehicle cost: 100000)
```

```
print(Vehicle.all)
```

```
[<__main__.Vehicle object at 0x000001839B26F0E0>, <__main__.Vehicle object at 0x000001839BD73750>, <__main__.Vehicle object at 0x000001839BD73D90>]
```

Object Oriented Programming

Object list storage and its representation:

`__repr__` : developer-friendly representation, ideally unambiguous and useful for debugging.

```
class Vehicle:
    company_name = "Tata Motors" # Class attribute
    all = []

    def __init__(self, color, price):
        self.color = color # Instance attribute
        self.price = price # Instance attribute

        Vehicle.all.append(self)

    def show_details(self):
        print(f"Company: {self.company_name}")
        print(f"Vehicle Color: {self.color}")
        print(f"Vehicle Price: {self.price}")

    # def __str__(self):
    #     return f"(Vehicle Color: {self.color}, vehicle cost: {self.price})"

    def __repr__(self):
        return f"Vehicle({self.color!r}, {self.price!r})"

v1=Vehicle("Black",100000)
v2=Vehicle("Red",200000)
v3=Vehicle("Brown",50000)
```

```
print(v1)
```

```
Vehicle('Black', 100000)
```

```
print(Vehicle.all)
```

```
[Vehicle('Black', 100000), Vehicle('Red', 200000), Vehicle('Brown', 50000)]
```

Object Oriented Programming

Object list storage and its representation:

```
class Vehicle:
    company_name = "Tata Motors" # Class attribute
    all = []

    def __init__(self, color, price):
        self.color = color # Instance attribute
        self.price = price # Instance attribute

        Vehicle.all.append(self)

    def show_details(self):
        print(f"Company: {self.company_name}")
        print(f"Vehicle Color: {self.color}")
        print(f"Vehicle Price: {self.price}")

    def __str__(self):
        return f"(Vehicle Color: {self.color}, vehicle cost: {self.price})"

    def __repr__(self):
        return f"Vehicle({self.color!r}, {self.price!r})"

v1=Vehicle("Black",100000)
v2=Vehicle("Red",200000)
v3=Vehicle("Brown",50000)
```

```
print(v1)
(Vehicle Color: Black, vehicle cost: 100000)
```

```
print(Vehicle.all)
[Vehicle('Black', 100000), Vehicle('Red', 200000), Vehicle('Brown', 50000)]
```

Code	Method used
<code>print(v1)</code>	<code>__str__</code>
<code>str(v1)</code>	<code>__str__</code>
<code>repr(v1)</code>	<code>__repr__</code>
<code>print([v1])</code>	<code>__repr__</code>
<code>print(Vehicle.all)</code>	<code>__repr__</code>



Object Oriented Programming

Dunder Methods

- Dunder methods are special methods in Python whose names start and end with double underscores (`__`).
- They tell Python how your objects should behave when you use normal Python operations.

<code>__init__()</code>	Creating/initializing an object	<code>v = Vehicle(...)</code>
<code>__str__()</code>	Human-readable string	<code>print(v)</code>
<code>__repr__()</code>	Developer representation	<code>repr(v)</code>
<code>__eq__()</code>	Equality	<code>v1 == v2</code>
<code>__ne__()</code>	Not equal	<code>v1 != v2</code>
<code>__lt__()</code>	Less than	<code>v1 < v2</code>
<code>__gt__()</code>	Greater than	<code>v1 > v2</code>
<code>__add__()</code>	Addition	<code>v1 + v2</code>
<code>__len__()</code>	<code>len()</code> behavior	<code>len(obj)</code>
<code>__getitem__()</code>	Indexing	<code>obj[0]</code>
<code>__setitem__()</code>	Assigning by index	<code>obj[0] = value</code>
<code>__call__()</code>	Calling an object	<code>obj()</code>

Object Oriented Programming

Object list storage and its representation:

```
class Vehicle:
    company_name = "Tata Motors"    # Class attribute
    all=[]

    def __init__(self, color, price):
        self.color = color          # instance attribute
        self.price = price          # instance attribute

        Vehicle.all.append(self)

    def show_details(self):
        print(f"Company: {self.company_name}")
        print(f"Vehicle Color: {self.color}")
        print(f"Vehicle Price: {self.price}")

    def __repr__(self):
        return f"Vehicle('{self.color}', {self.price})"
    |
    def __str__(self):
        return f"Vehicle('{self.color}' vehicle costs ₹{self.price})"
```

```
v1 = Vehicle("Black", 100000)
v2 = Vehicle("Red", 50000)
v3 = Vehicle("Black", 100000)
```

```
print(v1==v3)
```

False

```
: class Vehicle:
    company_name = "Tata Motors"    # Class attribute
    all=[]

    def __init__(self, color, price):
        self.color = color          # instance attribute
        self.price = price          # instance attribute

        Vehicle.all.append(self)

    def show_details(self):
        print(f"Company: {self.company_name}")
        print(f"Vehicle Color: {self.color}")
        print(f"Vehicle Price: {self.price}")

    def __str__(self):
        return f"Vehicle('{self.color}' vehicle costs ₹{self.price})"

    def __eq__(self, other):
        return self.color==other.color and self.price==other.price
```

```
print(v1==v3)
```

True

Object Oriented Programming

Destructor in Python:

- A Destructor is a special method named `__del__` that is automatically called when an object is about to be destroyed (deleted from memory).
- It is used to:
 - Clean up resources
 - Close files
 - Release memory
 - Disconnect database/network

```
class Vehicle:
    company_name = "Tata Motors" # Class attribute

    def __init__(self, color, price):
        self.color = color        # Instance attribute
        self.price = price        # Instance attribute

    def show_details(self):
        print(f"Company: {self.company_name}")
        print(f"Vehicle Color: {self.color}")
        print(f"Vehicle Price: {self.price}")

    def __del__(self):
        print("Destructor executed")

v1 = Vehicle("Black", 100000)
v2 = Vehicle("Red", 50000)

v1.show_details()
v2.show_details()

del v1
del v2
```

```
Company: Tata Motors
Vehicle Color: Black
Vehicle Price: 100000
Company: Tata Motors
Vehicle Color: Red
Vehicle Price: 50000
Destructor executed
Destructor executed
```

Object Oriented Programming

Lifecycle of an Object:

Lifecycle of an Object in Python

